Agenda:-

=======

-> What is DevSecOps?
-> Key Principles of DevSecOps
-> Benefits of DevSecOps
-> Processes used in DevSecOps
-> Static Application Security Testing (SAST)
-> Dynamic Application Security Testing (DAST)
-> Software Composition Analysis (SCA)
-> Infrastructure as Code (IaC)
-> DevSecOps Maturity Model
-> Performing DevSecOps
-> Merits and Demerits of DevSecOps
-> Real time issues and mitigations
-> OWASP (Open Web Application Security Project)
-> CIS (Center for Internet Security)
-> CIS Benchmarks
-> CIS Controls
-> What is CVE?
-> What is CVSS?
-> What is CWE?
-> What is CISA?

What is DevSecOps?

DevSecOps, short for Development, Security, and Operations, is an approach to software development that integrates security practices into the DevOps process. The goal is to build security into every phase of the software development lifecycle, from initial design through development, testing, deployment, and maintenance.
This approach ensures that security is a shared responsibility across all teams involved in the development and operations process.

The practice of integrating security practices within the DevOps process. This ensures that security is a shared responsibility throughout the entire IT lifecycle. DevSecOps emphasizes collaboration between development, security, and operations teams.

Example: Implementing automated security testing at every stage of the CI/CD

Key Principles of DevSecOps

Shift Left Security: Integrating security early in the development process, rather than as an afterthought, to identify and address vulnerabilities as soon as possible. The practice of testing for security vulnerabilities early in the development process to find and fix issues before they reach production. This helps to catch security flaws when they are easier and cheaper to fix.

Example: Integrating SAST tools like SonarQube into the CI pipeline to scan code for vulnerabilities as developers commit changes.

Collaboration: Encouraging close collaboration between development, security, and operations teams to ensure that security considerations are integrated into every stage of the development process

Continuous Monitoring: Continuously monitoring applications and infrastructure for security threats and vulnerabilities, even after deployment.
Compliance as Code: Defining and automating compliance policies to ensure that applications and infrastructure meet security and regulatory requirements.
Infrastructure as Code (IaC): The process of managing and provisioning computing infrastructure through machine-readable definition files, rather than physical hardware configuration or interactive configuration tools. IaC is a key DevOps practice that allows teams to automate infrastructure provisioning and management.
Example: Using Terraform scripts to provision and manage cloud resources in AWS.

Automation: Using automated tools to perform security testing, compliance checks, and vulnerability assessments as part of the continuous integration and continuous deployment (CI/CD) pipeline.
CI/CD (Continuous Integration/Continuous Deployment): A method to frequently deliver apps to customers by introducing automation into the stages of app development. CI/CD bridges the gaps between development and operations activities and teams by enforcing automation in building, testing, and deploying applications.
Example: Using Jenkins to automate the build, test, and deployment processes of

a microservices application.

Benefits of DevSecOps

1.**Early Detection of Vulnerabilities:** By integrating security into the early stages of the development process, vulnerabilities can be identified and addressed before they become critical issues.

2.**Reduced Costs:** Fixing security issues early in the development process is often less costly than addressing them after deployment.

3.**Faster Time to Market:** Automation and continuous monitoring help to streamline the development process, allowing for faster and more secure releases.

4.**Improved Security Posture:** Continuous monitoring and proactive security measures lead to a more robust and secure application and infrastructure.

1. Static Application Security Testing (SAST)

Definition:
SAST is a white-box testing method that analyzes source code, bytecode, or binary code for security vulnerabilities without executing the program. It is performed early in the development lifecycle.
Key Features:
Detects security issues in the code before the application is run. Can be integrated into the development environment (IDE) to provide real-time feedback to developers. Identifies vulnerabilities such as SQL injection, cross-site scripting (XSS), and buffer overflows.
Using SonarQube to scan the source code for vulnerabilities before the application is built and deployed.
Examples of SAST Tools:
SonarQube
Checkmarx
Fortify
Static Code Analyzer
Veracode

2. Dynamic Application Security Testing (DAST)

Definition:
DAST is a black-box testing method that analyzes a running application to identify security vulnerabilities. It simulates external attacks on the application to find weaknesses.
Key Features:
Tests the application in its running state.Does not require access to the source code.Identifies runtime vulnerabilities such as authentication issues, server misconfigurations, and SQL injection.
Examples of DAST Tools:
OWASP ZAP (Zed Attack Proxy) to scan a web application for vulnerabilities while it is running in a test environment.
Burp Suite
Netsparker
Acunetix

3. Software Composition Analysis (SCA)

Definition:
SCA tools analyze the third-party components and libraries used in an application to identify known vulnerabilities, license compliance issues, and outdated dependencies.
Key Features:
Scans open-source and third-party libraries.Provides information on known vulnerabilities (e.g., CVEs).Helps manage licenses to ensure compliance with legal and regulatory requirements.

Examples of SCA Tools:
Snyk
WhiteSource (Now it is called MEND)
Black Duck
Dependency-Check

4. Infrastructure as Code (IaC)

Definition:
IaC is the practice of managing and provisioning computing infrastructure through machine-readable definition files, rather than through physical hardware configuration or interactive configuration tools.

Key Features:
Allows infrastructure to be versioned and treated like application code.
Enables automated and consistent provisioning of infrastructure.
Facilitates the use of DevOps practices in infrastructure management.

Examples of IaC Tools:
Terraform
Ansible
AWS CloudFormation
PuppetChef

Security in IaC:
Checkov: Analyzes Terraform, CloudFormation, and other IaC files for misconfigurations and policy violations.
tfsec: A static analysis security scanner for Terraform code.
kube-score: Checks Kubernetes manifests for best practices and security issues.

DevSecOps Maturity Model

A framework that helps organizations assess their current DevSecOps practices and identify areas for improvement.

Stages:
======
Initial: Ad-hoc security practices.
Managed: Basic security measures are implemented.
Defined: Security processes are defined and standardized.
Quantitatively Managed: Security performance is measured and controlled.

Optimizing: Continuous improvement of security practices.

Performing DevSecOps

Pipeline Execution Using Key Tools and Technologies:

1.**Version Control (GitHub):** Code is checked into a Git repository.
2.**Continuous Integration (Jenkins):** Jenkins automatically builds and tests the code. SAST tools like SonarQube are integrated into the CI pipeline to scan the code for vulnerabilities.
3.**Containerization (Docker):** Docker is used to create container images from the built code.
4.**Orchestration (Kubernetes):** Kubernetes is used to deploy and manage containerized applications.
5.**Infrastructure as Code (Terraform):** Terraform scripts are used to provision and manage infrastructure.
6.Security Testing: Integrate DAST tools like OWASP ZAP to test running applications for vulnerabilities.
7.Monitoring and Logging: Use tools like Prometheus and ELK Stack for monitoring and logging to detect and respond to incidents.

Scenario: Deploying a microservices application using a DevSecOps pipeline.
1) Version Control: Developers push code changes to a GitHub repository.
2) CI/CD Pipeline:
- ➜ Jenkins triggers a build and runs SAST using SonarQube.
- ➜ Docker builds container images.
- ➜ Jenkins deploys images to a Kubernetes cluster using Helm.
- ➜ Terraform provisions infrastructure components like load balancers and databases.
- ➜ OWASP ZAP runs DAST on the deployed application.

3) **Monitoring and Logging:** Prometheus monitors application performance, and ELK Stack collects and visualizes logs.

Merits and Demerits of DevSecOps

Merits:-

Improved Security: Integrating security throughout the development process ensures that vulnerabilities are identified and addressed early.

Faster Time-to-Market: Automated testing and deployment pipelines reduce manual intervention, speeding up the release cycle.
Enhanced Collaboration: Encourages collaboration between development, security, and operations teams.
Consistent Environments: Use of containers and IaC ensures consistent environments across development, testing, and production.

Demerits:-

Complexity: Implementing DevSecOps requires a significant cultural and technological shift, which can be complex and resource-intensive.
Tool Integration: Integrating various tools (CI/CD, security, monitoring) can be challenging and requires expertise.
Initial Costs: The initial setup of DevSecOps practices and tools can be costly.
Learning Curve: Team members need to acquire new skills and knowledge, which may require training and time.

Real time issues and mitigations

1) **Issue:** Security Vulnerabilities in Deployed Applications
Problem: Deployed applications may have undetected security vulnerabilities.
Mitigation: Implement continuous monitoring and automated security testing (SAST/DAST) in the CI/CD pipeline.

2) **Issue:** Configuration Drift
Problem: Differences between environments can lead to issues that are hard to debug.
Mitigation: Use Infrastructure as Code (IaC) to maintain consistent configurations across all environments.

3) **Issue:** Resource Management
Problem: Inefficient use of resources can lead to high costs and performance issues.
Mitigation: Implement resource quotas and limits in Kubernetes, and use monitoring tools like Prometheus to track resource usage.

4) **Issue:** Lack of Visibility
Problem: Lack of visibility into the deployment process and application performance.
Mitigation: Use logging and monitoring tools like ELK Stack and Prometheus to gain insights and quickly detect issues.

5) **Issue:** Compliance and Auditing
Problem: Ensuring compliance with industry standards and regulations.
Mitigation: Use tools like CIS Benchmarks and automate compliance checks in the CI/CD pipeline.

OWASP (Open Web Application Security Project)

OWASP ZAP (Zed Attack Proxy): An open-source web application security scanner that helps find security vulnerabilities in your web applications during development and testing.

Scenario: Using OWASP ZAP to perform automated security scans on a web application as part of the CI/CD pipeline to identify and remediate vulnerabilities before deployment.

OWASP Top 10: A standard awareness document for developers and web application security, representing a broad consensus about the most critical security risks to web applications.
Scenario: Educating development teams about the OWASP Top 10 vulnerabilities (e.g., SQL Injection, Cross-Site Scripting) and implementing secure coding practices to mitigate these risks.

OWASP Mobile Top 10: A list of the top 10 security risks for mobile applications.
Scenario: Using the OWASP Mobile Top 10 as a guideline to conduct security assessments on a new mobile banking application.

OWASP Mobile Top 10: A list of the top 10 security risks for mobile applications.
Scenario: Using the OWASP Mobile Top 10 as a guideline to conduct security assessments on a new mobile banking application.

OWASP Cheatsheet: A collection of concise, technical guidelines on specific security issues written by the OWASP community.
Scenario: Referencing the OWASP Cheatsheet for best practices on implementing secure password storage in an application.

CIS (Center for Internet Security)

CIS Benchmarks

Consensus-developed secure configuration guidelines for hardening operating systems, servers, cloud environments, and software.

Scenario: Using CIS Benchmarks to configure security settings for AWS cloud infrastructure, ensuring that instances are securely configured according to industry standards.

CIS Benchmarks provide security guidelines and best practices for securing IT systems and data. They are developed by the Center for Internet Security (CIS) through a collaborative process involving cybersecurity experts.

Standardized Security: They offer standardized security settings that organizations can apply to various systems, ensuring a consistent security posture.

Compliance: Many regulatory requirements and frameworks, like HIPAA, PCI-DSS, and GDPR, reference CIS Benchmarks. Using them helps organizations achieve and demonstrate compliance.

Reduced Vulnerabilities: By following the guidelines, organizations can reduce vulnerabilities in their systems, thereby lowering the risk of cyberattacks.

Cost-Effective: Implementing these benchmarks can be more cost-effective than developing custom security protocols, saving time and resources.

Documentation: They provide detailed documentation that helps IT teams understand the reasoning behind each recommendation, which can be useful for audits and training purposes.

CIS Controls:

A set of best practices for securing IT systems and data against the most pervasive attacks.

Scenario: Implementing CIS Controls to establish a robust security posture for an organization, including measures such as regular vulnerability assessments and secure configuration management.

CIS Controls are a prioritized set of actions developed by the Center for Internet Security (CIS) to protect organizations from known cybersecurity threats.

1)**Prioritized Approach:** CIS Controls are organized by priority, starting with the

most effective and foundational measures. This helps organizations focus on what will provide the most significant security impact first.

2. Comprehensive Coverage: The controls cover a wide range of security areas, including asset management, access control, and incident response, offering a holistic approach to cybersecurity.

3) Proven Effectiveness: The controls are based on actual threats and attack patterns, making them highly effective in real-world scenarios. They are continuously updated to reflect the latest threats.

4) Ease of Implementation: CIS Controls are designed to be actionable and straightforward, making it easier for organizations of all sizes to implement them, even with limited resources.

5) Alignment with Frameworks: Many organizations find that implementing CIS Controls helps them meet the requirements of other cybersecurity frameworks and regulations, such as NIST, ISO, and GDPR.

6) Community-Driven: They are developed and refined by a global community of IT professionals, cybersecurity experts, and academics, ensuring they are practical and up-to-date.

What is CVE?

CVEs (Common Vulnerabilities and Exposures): A list of publicly disclosed computer security flaws. It is a standardized identifier for known security vulnerabilities in software and hardware systems.
Scenario: Regularly monitoring CVE databases for new vulnerabilities affecting software components used in your applications and applying patches as needed.
1**) Unique Identifiers:** Each CVE entry is assigned a unique identifier, like "CVE-2024-12345."
2) Centralized Repository: The CVE database is managed by the MITRE Corporation, providing a centralized repository of known vulnerabilities.
3) Facilitates Communication: By providing a common language for discussing vulnerabilities, CVE enables better communication and coordination between different organizations, vendors, and security tools.
4) Part of a Larger Ecosystem: CVE is often used in conjunction with other security tools and frameworks, such as the National Vulnerability Database (NVD), which provides additional details like severity ratings (CVSS scores) and mitigation

guidance.
5) Critical for Patch Management: Organizations use CVE identifiers to track vulnerabilities in their systems and apply patches or other security measures as needed. This helps in prioritizing the most critical issues.

6) Supports Compliance and Reporting: Many regulatory and compliance frameworks require organizations to track and address CVEs as part of their security posture.

What is CVSS?

CVSS (Common Vulnerability Scoring System): A free and open industry standard for assessing the severity of computer system security vulnerabilities.
It is a standardized framework used to assess and communicate the severity of security vulnerabilities.
CVSS provides a numerical score that helps organizations prioritize which vulnerabilities to address first based on their potential impact.
Scenario: Using CVSS scores to prioritize remediation efforts based on the severity of discovered vulnerabilities. Scores range from 0.0 (no severity) to 10.0 (maximum severity).

CVSS is widely used in the cybersecurity industry, including by organizations such as the National Vulnerability Database (NVD) and various security tools, to help prioritize security efforts.

What is CWE?

CWE (Common Weakness Enumeration): A list of software weaknesses.
It is a community-developed list of common software and hardware security weaknesses. Unlike CVE, which focuses on specific vulnerabilities, CWE identifies the underlying flaws that can lead to vulnerabilities.

•**Scenario:** Using the CWE list to identify common coding errors and implement secure coding practices to avoid them.

CWE-79: Cross-Site Scripting (XSS)
This CWE refers to the weakness that allows attackers to inject malicious scripts into web pages viewed by other users.

CWE-120: Buffer Copy without Checking Size of Input (Classic Buffer Overflow)

This CWE addresses a common issue where a program does not properly validate the length of data before copying it into a buffer, leading to potential overflows.

What is CISA?

CISA (Cybersecurity and Infrastructure Security Agency): A standalone United States federal agency under the Department of Homeland Security that works to improve cybersecurity across all levels of government.
It is a U.S. government agency responsible for enhancing the security, resilience, and reliability of the nation's critical infrastructure.
Scenario: Leveraging CISA alerts and guidelines to stay informed about the latest cybersecurity threats and best practices for mitigating them.
1) Cybersecurity
2) Infrastructure Security
3) Election Security
CISA's mission is to protect the nation's critical infrastructure from physical and cyber threats, making it a cornerstone of the U.S. government's efforts to secure the homeland.